面向边缘 AI 大语言模型推理的 RISC-V 自定义 vdot 指令设计——基于“香山”昆明湖 V2 处理器架构

第二阶段竞赛报告

设计文档与基础测试

目标平台 香山昆明湖 V2，VLEN=128

队伍 群贤芯跳

目录

一 阶段目标与完成情况 3
二 指令方案与运算定义 4
三 香山集成与 RTL 实现 6
四 功能验证与测试结果 9
五 阶段成果与后续完善 11

一 阶段目标与完成情况

我们在香山昆明湖 V2 的现有向量执行框架中新增了 vdot.vv 自定义指令，完成八项 signed int8 乘积归约和 32 位结果写回。实现覆盖指令编码、向量译码、功能单元配置、Chisel 数据通路、生成 RTL 与独立 testbench，形成了从指令定义到基础验证的完整交付。

本阶段完成 SEW=8、VLEN=128 条件下的点积设计与 RTL 实现，并以正数、负数及边界输入开展基础功能测试。

我们交付了完整香山工程源码与改动文件、生成 SystemVerilog、独立组合 RTL 和自检 testbench。五组定向测试覆盖正数、负数、混合符号、较大正和及最小和边界。

五组独立 RTL 定向比较全部通过，运行日志为 checks=5 errors=0。测试逐项检查完整 128 位输出，验证了低 32 位点积结果和高 96 位清零格式。

方案复用了香山已有的向量调度、源操作数接口和写回通路，将新增运算集中在独立功能单元中。实现通过显式有符号转换、扩位归约和固定结果布局完成了数据语义与执行接口的对应。

二 指令方案与运算定义

2.1 e8 输入与结果布局

输入范围与结果格式

我们将源元素宽度设为 SEW=8，并固定选择两个 128 位源向量的低八个 byte，即 vs1[63:0] 与 vs2[63:0]。每项按 signed int8 参与乘法，八项乘积求和后写入目的向量低 32 位。



图 1 当前 e8 源元素与 32 位结果布局

图 1 给出了实现采用的数据布局：byte 0 位于最低八位，byte 7 位于 [63:56]。源元素按 e8 解释，目的向量以低 32 位二补码保存归约值，高 96 位由功能单元统一清零。

```
for k = 0..7:
    p[k] = signed8(vs1[8*k +: 8]) * signed8(vs2[8*k +: 8])
result = sum(p[k])
vd[31:0]   = result[31:0]
vd[127:32] = 0
```

2.2 有符号运算与位宽

字节符号解释

我们分别在 Chisel 和 Verilog 中保留了源字节的符号属性。Scala 切片通过 .asSInt 转换，独立 RTL 将输入切片保存到 signed [7:0] 变量，使 0x80 与 0xff 分别按 -128 和 -1 参与运算。

```
val a = vs1(8 * k + 7, 8 * k).asSInt
val b = vs2(8 * k + 7, 8 * k).asSInt
products(k) := a * b
```

项	数值范围或位宽	设计意义
单个 int8 源元素	-128 至 127，8 位	必须进行有符号解释
单个乘积	-16256 至 16384，16 位可容纳	最大正积来自 (-128)×(-128)
八项和	-130048 至 131072	最大值等于 2^17，至少需 19 位有符号表示

项	数值范围或位宽	设计意义
Chisel 存储	products 为 SInt(18.W)，先 pad(21)，再 +& 归约	18 位乘积与扩位加法留有余量；最终 sumWide 不应简单称为固定 21 位
架构写回	低 32 位结果，高 96 位零	负结果只在低 32 位保留二补码

负数测试验证了结果布局。八项 -128×1 的和为 -1024，低 32 位为 0xfffffc00，完整向量输出为 00000000000000000000000ffffc00。Chisel 的 Cat 与独立 RTL 的拼接语句共同实现该写回格式。

2.3 自定义编码与译码

编码字段

我们在 OP-V 空间分配 funct6=101100、funct3=010、opcode=1010111，并在香山译码表中建立对应入口。系统测试采用 vm=1，以 .word 发射自定义机器码，完成了软件测试程序与硬件译码的衔接。

字段	位段	当前值	含义
funct6	31:26	101100	标识自定义点积操作
vm	25	1 用于当前测试	非掩码形式
vs2	24:20	指令寄存器字段	第二源向量
vs1	19:15	指令寄存器字段	第一源向量
funct3	14:12	010	OPMVV 编码组
vd	11:7	指令寄存器字段	目标向量
opcode	6:0	1010111	OP-V

```
def VDOT_VV = BitPat("b101100??????????010?????1010111")

// tests/system/vdot_exact_test.S 的宏参数顺序：vd, vs1, vs2
.word ((0x59 << 25) | (vs2 << 20) | (vs1 << 15) |
       (2 << 12) | (vd << 7) | 0x57)

VDOT 8, 6, 4  ->  vs2=4, vs1=6, vd=8  ->  0xB2432457
```

宏参数与机器码字段已逐项对应；点积的两个源交换后具有相同数学结果，测试仍按字段定义组织寄存器。

三 香山集成与 RTL 实现

3.1 向量执行路径接入

一 从指令到功能单元的完整链

```
Instructions.scala 识别 VDOT_VV
    -> VecDecoder 输出 FuType.vdot
    -> FuType / FuConfig 提供分类、接口与固定延迟
    -> Parameters 的 VFEX0 挂载 VdotCfg
    -> VDotU 计算并经 res.data 输出
    -> 复用向量寄存器写回路径
```

完整文件位置	关键修改和作用
rocket-chip/src/main/scala/rocket/Instructions.scala:408	定义 VDOT_VV 的 BitPat
backend/decode/VecDecoder.scala:384	VDOT_VV 映射到 FuType.vdot，构造普通向量源形式
backend/fu/FuType.scala:64、128、265	添加 vdot 类型，加入 vecOPI，设置名称映射
backend/fu/FuConfig.scala:566	VdotCfg 绑定 new VDotU，声明向量写回和 illegalInstr
src/main/scala/xiangshan/Parameters.scala:448	在 VFEX0 的功能单元序列加入 VdotCfg
backend/fu/wrapper/VDotU.scala	实现八字节有符号点积、结果寄存和 SEW 检查

二 配置接口与实际数据依赖

```
srcData = Seq(Seq(VecData(), VecData(), VecData(),
                  V0Data(), VlData()))
piped = true
writeVecRf = true
latency = CertainLatency(2)
destDataBits = 128
exceptionOut = Seq(illegalInstr)
```

我们沿用向量执行单元的通用输入接口，并在 VDotU 中明确选择 vs1、vs2 作为算术源。当前数据通路执行固定八项归约，旧 vd、掩码合并和通用向量参数处理不参与该算术实现；相关扩展条件集中列在验证范围中。

3.2 Chisel 数据通路

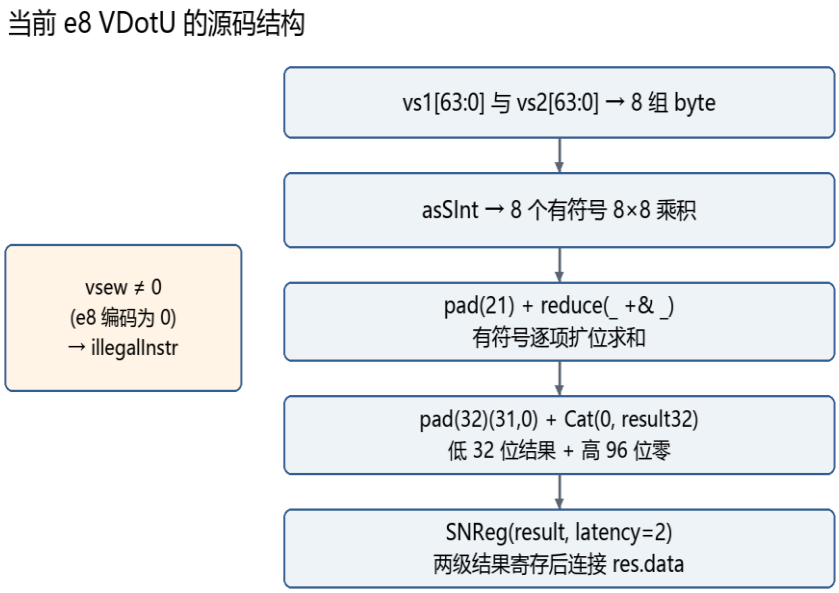


图 2 当前 e8 数据通路及结果寄存结构

Chisel 数据通路将低 64 位切成八对字节，通过 .asSInt 恢复符号，形成八路 products；扩位加法完成 sumWide 归约，再拼接低 32 位结果与高位零，最后经 SNReg 接入 res.data 写回通路。

3.3 功能单元核心实现

```
private val products = Wire(Vec(8, SInt(18.W)))
for (k <- 0 until 8) {
  val a = vs1(8 * k + 7, 8 * k).asSInt
  val b = vs2(8 * k + 7, 8 * k).asSInt
  products(k) := a * b
}

private val sumWide = products.map(_.pad(21)).reduce(_ +& _)
private val result32 = sumWide.pad(32)(31, 0).asUInt
private val result = Cat(0.U((dataWidth - 32).W), result32)

private val resultPiped = SNReg(result, latency)
io.out.bits.res.data := resultPiped

io.out.bits.ctrl.exceptionVec.get(ExceptionNO.illegalInstr) :=
  outVecCtrl.vsew =/= 0.U
```

代码	实现内容与作用
Vec(8, SInt(18.W))	明确只有 8 个乘积；这与 VLEN 可容纳 16 个 e8 元素不同
8*k+7, 8*k	k=0 对应最低字节；k=7 截止 bit63，高 64 位没有进入乘法
.asSInt	用源字节的 bit7 恢复符号，确保 0x80 为 -128
pad(21) 与 +&	先符号扩展，再进行会扩位的加法；不能用无符号零扩展替代
pad(32)(31,0) 与 Cat	保留 32 位二补码结果，再把其上方所有位设为零
SNReg 与 illegalInstr	前者属于结果寄存通路，后者检测非 e8 的配置

3.4 生成 RTL 与独立模型

生成 RTL 的运算对应

我们保存了香山构建生成的完整 VDotU.sv。生成物包含执行接口、乘法与归约逻辑、结果寄存器和异常输出，与 Chisel 源码一并交付；_products、_GEN 和 resultPiped 等信号用于对应各级运算。

```
// 生成物摘录，完整文件见 coding/overlay/XiangShan/build_exact/rtl/VDotU.sv
wire [15:0] _products_0_T =
  16'({{8{io_in_bits_data_src_0[7]}}, io_in_bits_data_src_0[7:0]}
      * {{8{io_in_bits_data_src_1[7]}}, io_in_bits_data_src_1[7:0]});

if (io_in_bits_validPipe_0 & io_in_valid & rdyVec_0)
  resultPiped_r <= {96'h0, {7{_GEN_5[24]}}, _GEN_5};
if (io_in_bits_validPipe_1 & validVecThisFu_1 & rdyVec_1)
  resultPiped <= resultPiped_r;

assign io_out_bits_ctrl_exceptionVec_2 = |io_in_bits_ctrlPipe_2_vpu_vsew;
assign io_out_bits_res_data = resultPiped;
```

我们对生成 RTL 的符号扩展、八项输入、归约结果、高位清零及寄存控制进行了源码对应。生成器对冗余位和中间信号的裁剪保持了上述运算与输出关系。

生成 RTL 与独立模型分工

文件	端口和职责	本阶段证据
VDotU.sv	香山接口，带时钟、valid/ready 和异常等	保存的生成物，供源码核对
vdot_exact_unit.v	两个 128 位输入、一个 128 位输出的组合数据通路	Icarus 编译、自检日志与 VCD

四 功能验证与测试结果

4.1 自检方法与检查点

```
always @* begin
    acc = 32'sd0;
    for (k = 0; k < 8; k = k + 1) begin
        a = vs1[8*k +: 8];
        b = vs2[8*k +: 8];
        prod = a * b;
        acc = acc + {{16{prod[15]}}, prod};
    end
end
assign vd = {96'd0, acc};
```

独立 RTL 采用 signed 8 位源、signed 16 位乘积和 signed 32 位累加变量。每次组合求值从零开始，对八个乘积逐项符号扩展后求和，并以 {96'd0, acc} 形成完整输出。

```
vs1 = a;
vs2 = b;
#1;
checks = checks + 1;
if (vd !== {96'd0, expected}) begin
    errors = errors + 1;
    $display("[FAIL] expected=%08h got=%032h", expected, vd);
end
```

testbench 对完整 128 位结果进行四态比较，同时检查数值、高位清零和 X/Z 状态。失败用例增加 errors，末尾通过 $fatal 返回失败状态，形成可由脚本调用的基础自检流程。

4.2 五组基础测试结果

用例	低 64 位输入摘要	计算与期望	覆盖点
正数	vs1=01…01；vs2=02…02	8×(1×2)=16 00000010	基本八项归约
负数	vs1=80…80；vs2=01…01	8×(-128×1)=-1024 fffffc00	负源符号扩展
混合符号	vs1=0000000001ff0201 vs2=000000000103ff02	1×2+2×(-1)+(-1)×3+1×1=-2 fffffffe	不同 byte 的位置和符号
较大正值	vs1=7f…7f；vs2=7f…7f	8×(127×127)=129032 0001f808	较大正和
最小和边界	vs1=80…80；vs2=7f…7f	8×(-128×127)=-130048 fffe0400	最小八项和

混合符号用例按最低 byte 起逐项配对，得到 -2。该用例与阶段三系统汇编中的另一组混合符号输入分别保留，构成两组输入不同、期望结果相同的定向检查。

本阶段的较大正值用例采用八项 127×127，得到 129032；最小和边界采用八项 -128×127，得到 -130048。八项 -128×-128 对应的最大正和 131072 由阶段三系统测试补充覆盖。

独立 RTL 自检原始日志

```
[PASS] expected=00000010 result=00000000000000000000000000000010
[PASS] expected=fffffc00 result=00000000000000000000000fffffc00
[PASS] expected=fffffffe result=00000000000000000000000ffffffe
[PASS] expected=0001f808 result=00000000000000000000000001f808
[PASS] expected=fffe0400 result=00000000000000000000000fffe0400
checks=5 errors=0
```

图 3 独立 RTL 实际自检日志

图 3 记录了五条 PASS 和 checks=5 errors=0，分别对应表中的五组输入。独立组合测试完成了基础算术及完整结果格式验证，流水握手和系统异常测试归入后续集成验证范围。

4.3 波形数值验证

我们通过独立组合 testbench 生成并保存了 e8 VCD，将五组输入的 vs1、vs2、vd 和 expected 逐组对应。波形显示八项 1×2 的低 word 为 0x00000010，八项 -128×1 为 0xFFFFFC00，各组高 96 位均为零。

当前 e8 独立组合 RTL 的 VCD 数值图

从真实 vdot_exact_wave.vcd 提取；每格为一组输入的保持区间，时间单位 ns

	0–1 ns	1–2 ns	2–3 ns	3–4 ns	4–5 ns
vs1[63:0]	0101010101010101	8080808080808080	0000000001FF0201	7F7F7F7F7F7F7F7F	8080808080808080
vs2[63:0]	0202020202020202	0101010101010101	000000000103FF02	7F7F7F7F7F7F7F7F	7F7F7F7F7F7F7F7F
vd[31:0]	00000010	FFFFFC00	FFFFFFFE	0001F808	FFFE0400
vd[127:32]	000000…0000	000000…0000	000000…0000	000000…0000	000000…0000
expected[31:0]	00000010	FFFFFC00	FFFFFFFE	0001F808	FFFE0400

对应结果：16　-1024　-2　129032　-130048

该文件没有 clk、reset、valid 或 ready。#1 是 testbench 的观测间隔，不是 CPU 指令延迟。

图 4 独立 e8 组合模型 VCD 的输入与结果对应

该波形记录组合运算的数值响应，激励间隔为 testbench 的 1 ns 设置。我们以它验证符号、byte 位置和写回格式；核级流水与异常行为列入集成验证范围。

五 阶段成果与后续完善

我们完成了自定义编码、香山译码和功能单元挂载，实现八项有符号乘积归约及固定结果写回，并以五组定向输入通过完整 128 位比较。源码、生成 RTL、自检程序及波形材料构成本阶段的主要成果。下表汇总已完成的验证与后续扩展条件。

项目	当前证据状态
基础算术及结果格式	五组定向输入已通过完整 128 位比较
SEW 合法性逻辑	Scala 与生成 RTL 中存在非 e8 的 illegalInstr 判断；本阶段数据 testbench 不覆盖其端口
掩码 vm=0	当前算术没有 Mgu 掩码合并；不列为已验证功能
vl、vstart 与 LMUL	当前 VDotU 的计算固定取低八项；本次测试不证明任意配置的 RVV 语义
源高 64 位不影响结果	代码只切取 [63:0]；现有五组输入高位为零，还可增加高位非零的专门回归
随机输入与最大正和	独立五组未覆盖全部空间；建议增加最大正和 131072 和随机对照
流水及整核异常	需要带时钟和握手信号的集成仿真或系统级 trap 测试
面积、频率与功耗	本阶段未提供综合和实现报告，不作确定数值结论

本阶段完成了固定八项 signed int8 点积的指令接入、数据通路与基础验证。后续工作围绕随机数据、参数扩展及集成流水与异常行为展开，在保持当前 32 位结果契约的基础上扩大验证覆盖。

附录 A 测试复现入口

我们将独立 RTL、基础 testbench、波形 testbench 和运行脚本集中到 coding 目录，形成 Linux 环境下的复现入口。下列命令与提交目录结构对应。

```
iverilog -g2012 -o /tmp/vdot_e8_test.vvp \
  coding/rtl/standalone/vdot_exact_unit.v \
  coding/tests/rtl/tb_vdot_exact.v
vvp /tmp/vdot_e8_test.vvp

# 同时运行基础自检和波形自检，并保留输出路径
bash coding/scripts/run_rtl_tests.sh
```

波形 testbench 使用相对路径输出 vdot_exact_wave.vcd。我们在独立运行目录执行该程序并保存 VCD，与自检日志和数值图一起归档。

```
iverilog -g2012 -o /tmp/vdot_e8_wave.vvp \
  coding/rtl/standalone/vdot_exact_unit.v \
  coding/tests/rtl/tb_vdot_exact_wave.v
(cd /tmp && vvp ./vdot_e8_wave.vvp)
gtkwave /tmp/vdot_exact_wave.vcd
```

证据文件	复核内容
coding/evidence/logs/vdot_exact_sim.log	原始五组 PASS 与 checks=5 errors=0
coding/evidence/logs/run_rtl_packaged_20261108.log	提交脚本在 VM 内重新运行的记录
coding/evidence/waveforms/vdot_exact_wave.vcd	组合单元实际输入、输出和期望值
阶段 2/png/vdot_e8_vcd_values_actual.png	从实际 VCD 整理的可读数值图

附录 B 源码与文件索引

交付源码 VDotU.scala 完整内容

来源：coding/overlay/XiangShan/.../wrapper/VDotU.scala

```
package xiangshan.backend.fu.wrapper

import org.chipsalliance.cde.config.Parameters
import chisel3._
import chisel3.util._
import xiangshan.backend.fu.FuConfig
import xiangshan.backend.fu.vector.VecPipedFuncUnit
import xiangshan.ExceptionNO

/**
  * Exact competition semantics for vdot.vv (SEW=8, VLEN=128):
  *
  *   result = sum(k=0..7) sext(vs1.byte[k]) * sext(vs2.byte[k])
  *   vd[31:0] = result[31:0]
  *   vd[127:32] = 0
  *
  * The operation is a reduction. It does not read old vd, does not perform
  * per-element accumulation, and only accepts vtype.vsew=e8.
  */
class VDotU(cfg: FuConfig)(implicit p: Parameters) extends VecPipedFuncUnit(cfg) {
  private val dataWidth = cfg.destDataBits

  require(dataWidth >= 32)

  private val products = Wire(Vec(8, SInt(18.W)))
  for (k <- 0 until 8) {
    val a = vs1(8 * k + 7, 8 * k).asSInt
    val b = vs2(8 * k + 7, 8 * k).asSInt
    products(k) := a * b
  }

  private val sumWide = products.map(_.pad(21)).reduce(_ +& _)
  private val result32 = sumWide.pad(32)(31, 0).asUInt
  private val result = Cat(0.U((dataWidth - 32).W), result32)

  private val resultPiped = SNReg(result, latency)
  io.out.bits.res.data := resultPiped

  io.out.bits.ctrl.exceptionVec.get(ExceptionNO.illegalInstr) :=
    outVecCtrl.vsew =/= 0.U
}
```

图 5 当前 VDotU.scala 完整清单，按交付源码生成

我们将完整 VDotU.scala 作为源码附录交付，覆盖类定义、参数约束、运算、寄存器和异常输出；正文保留关键代码及其与数据通路的对应说明。

完整文件索引

交付文件按完整工程、修改文件副本、独立 RTL 和测试程序分类保存，位置如下。

文件	用途
coding/overlay/XiangShan/rocket-chip/src/main/scala/rocket/Instructions.scala	完整编码定义
coding/overlay/XiangShan/src/main/scala/xiangshan/Parameters.scala	VFEX0 挂载
coding/overlay/XiangShan/src/main/scala/xiangshan/backend/decode/VecDecoder.scala	译码输出
coding/overlay/XiangShan/src/main/scala/xiangshan/backend/fu/FuType.scala	功能单元分类
coding/overlay/XiangShan/src/main/scala/xiangshan/backend/fu/FuConfig.scala	执行接口与固定延迟配置
coding/overlay/XiangShan/src/main/scala/xiangshan/backend/fu/wrapper/VDotU.scala	Chisel 数据通路
coding/overlay/XiangShan/build_exact/rtl/VDotU.sv	香山生成 SystemVerilog
coding/rtl/standalone/vdot_exact_unit.v	独立组合 RTL
coding/tests/rtl/tb_vdot_exact.v	五组基础自检
coding/tests/rtl/tb_vdot_exact_wave.v	相同数据用例与 VCD 输出
coding/scripts/run_rtl_tests.sh	可运行的独立回归入口